

קבצים

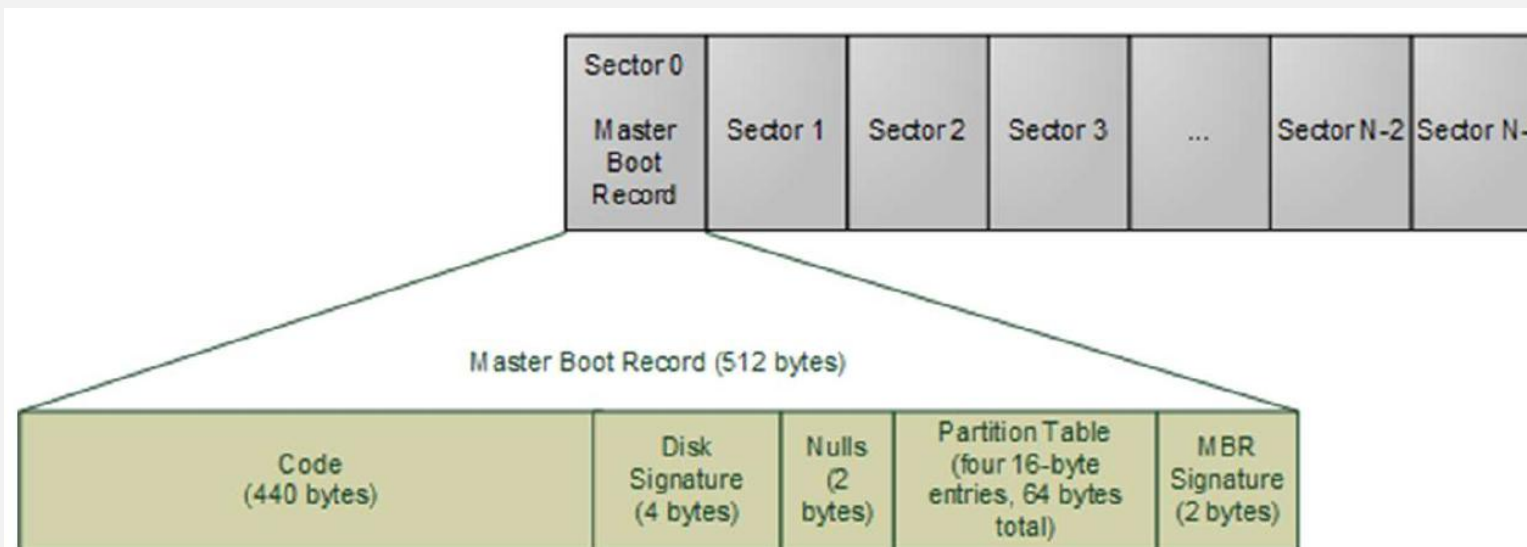
מערכות הפעלה
לינוקס

מערכות קבצים

- בזמן ריצת תהליך, יש לו זיכרון מוגבל לשימוש
- עבור תהליכים רבים זה לא מספיק לריצתם התקינה
- בנוסף תהליכים רבים מעוניינים לשמור או לקרוא מידע מחוץ לריצה שלהם, ותהליכים שונים לעיתים צריכים לגשת לאותם נתונים
- לשם כך מערכות הפעלה מארגנות את הנתונים במערכת קבצים
- קבצים הם יחידות לוגיות של מידע הנשמרים בהתקני זיכרון קבועים (קוננים)
- יש מערכות קבצים רבות. נפוצות בינהן:
 - NTFS עבור windows
 - EXT עבור Linux

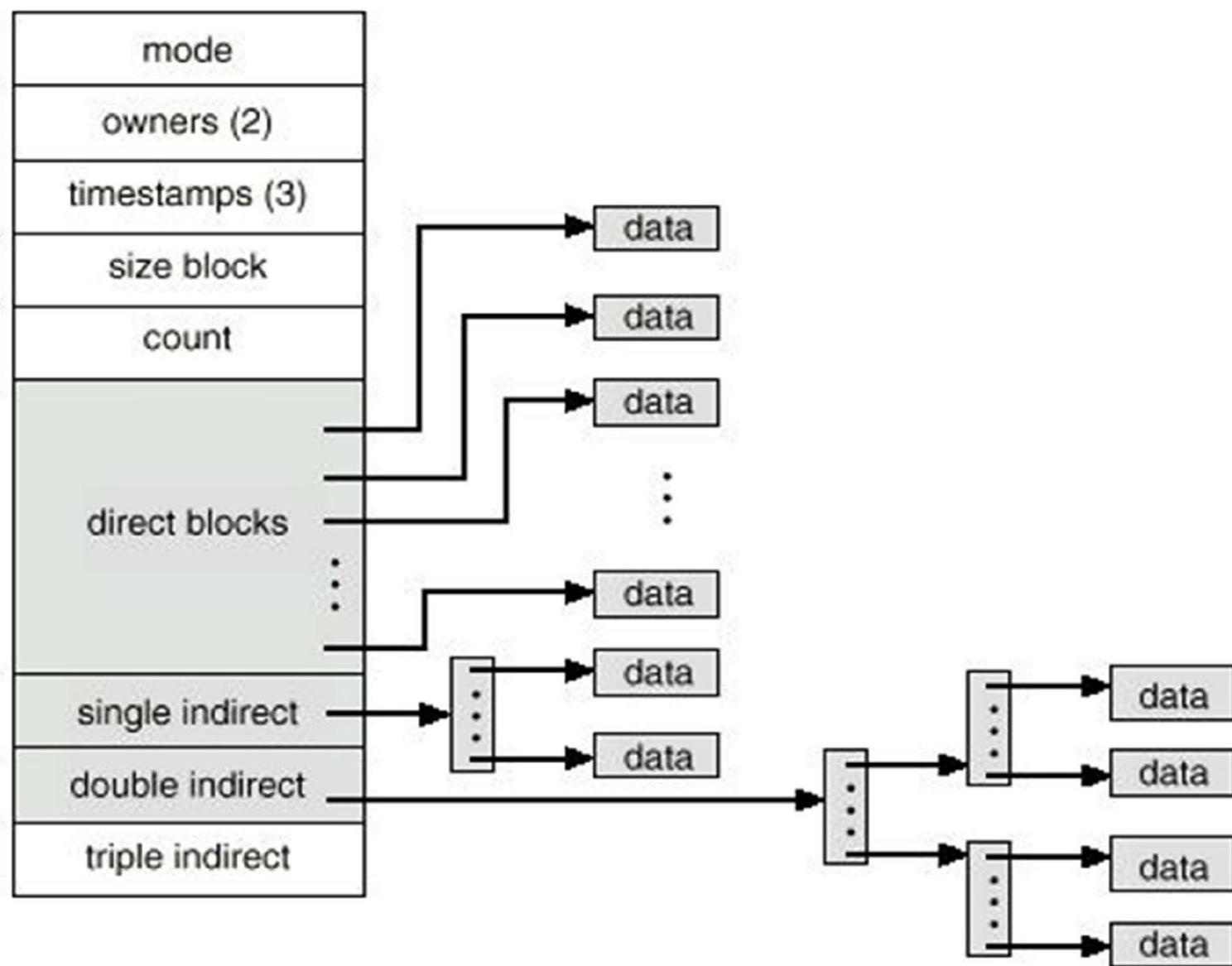
מערכות קבצים

- קוננים לרוב מחולקים למחיצות, כאשר כל מחיצה יכולה להשתמש במערכת קבצים שונה
- כך למשל, ניתן להתקין על אותו קונן שתי מערכות הפעל שונות עם שתי מערכות קבים שונות
- סקטור (sector) היא יחידת המידה הקטנה ביותר על הדיסק
- מערכת הקבצים מגדירה גודל בלוק (block) קבוע שהוא כפולה של סקטורים (אחד או יותר
- הסקטור הראשון בקונן נקרא MBR (Master Boot Record) אשר מכיל מידע על חלוקת המחיצות בקונן, ומידע שהמחשב צריך כדי להריץ את מערכת ההפעלה



EXT

- מערכות Linux משתמשות במערכת הקבצים EXT
- עובד עם מבני נתונים הנקראים inode (index-node)
- כל inode שומר מידע עבור קובץ או תיקייה
- inode לא מכיל את הנתונים באופן ישיר אלא מצביע לכתובות של בלוקים בהם הנתונים שמורים
- inode יש גודל קבוע מה שמאפשר גישה מהירה לפי אינדקס
- inode שומר נתונים שונים עבור הקובץ, ומצביעים לבלוקים בזיכרון
- Direct blocks – רשימה של כתובות בלוקים
- Single indirect – כתובת של בלוק בזיכרון שמכיל את כתובות נוספות
- Double indirect – כתובת של בלוק המכיל כתובות של בלוקים המכילים כתובות
- Triple indirect – כתובת של בלוק המכיל כתובות של בלוקים המכילים כתובות של בלוקים המכילים כתובות



EXT

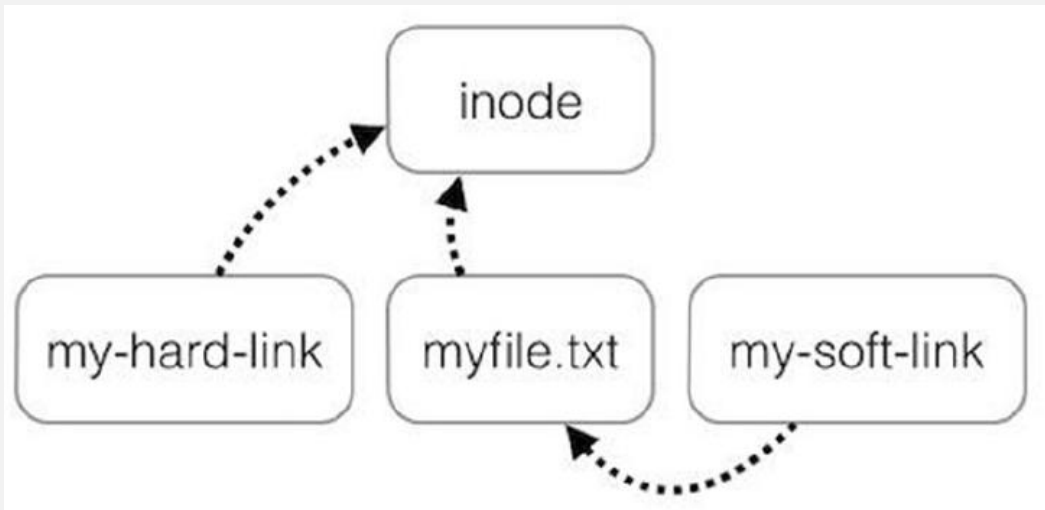
- תיקיות נשמרות של ידי רשימה של שמות שמקושרים למספר inode ששמור מידע עבור הקובץ

/usr/ast	
16	mail
81	games
40	test

- התיקיה הראשית במחיצה תמיד שמורה בinode מספר 2, ולכן תמיד ניתן לגשת אליה
- כאשר נוצרת תיקיה חדשה היא תמיד כוללת שתי תיקיות מיוחדות
 - . – התיקיה הנוכחית
 - .. – התיקיה שמכילה אותה

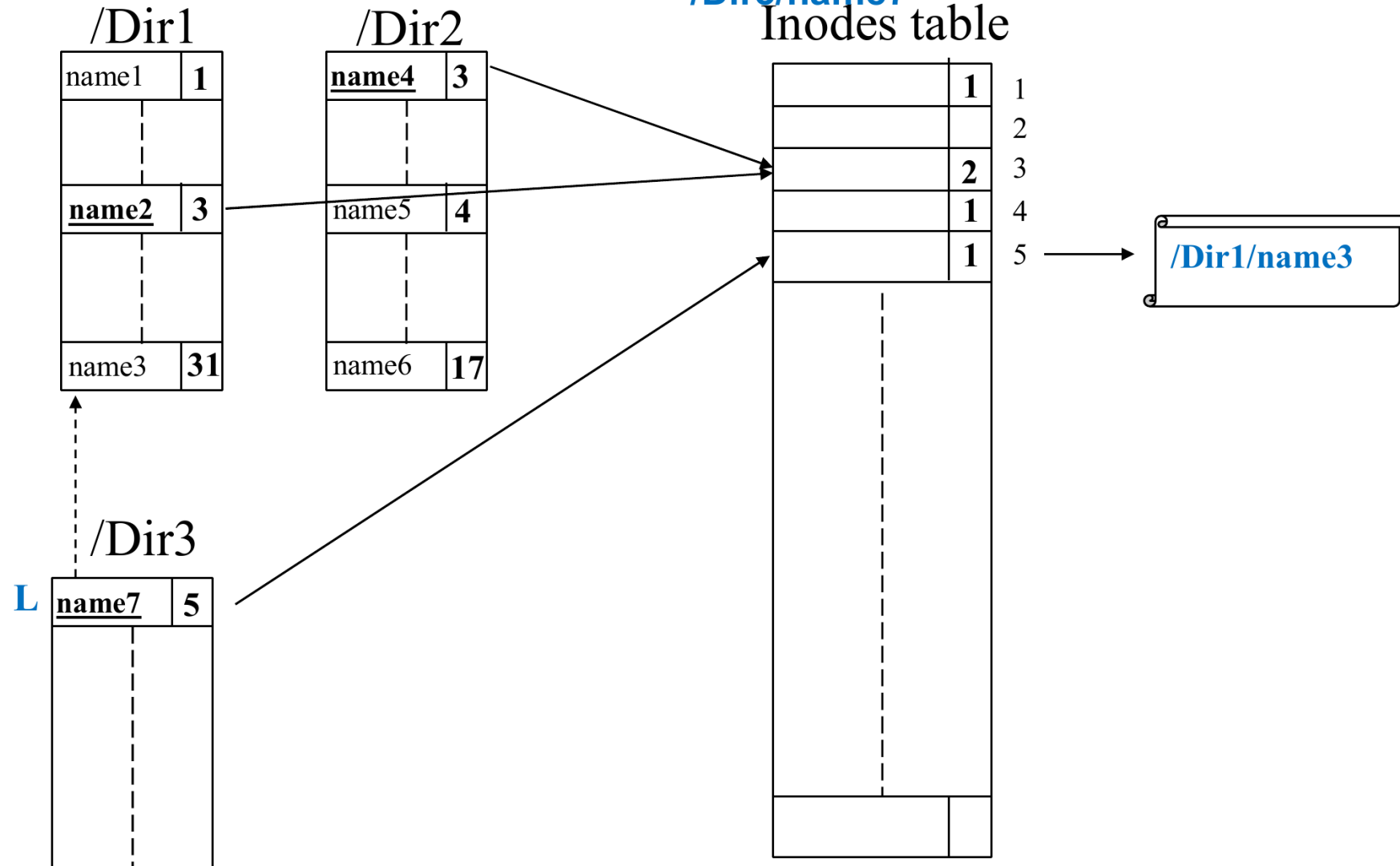
LINK

- ניתן לקשר קובץ שנמצא ביתקיה אחת לתיקיה אחרת בשני אופנים:
- **קישור חלש** (symbolic/soft link)
 - שמירת קובץ המכיל את הכתובת של הקובץ המקורי
 - פקודה `ln -s src dest`
- **קישור חזק** (hard link)
 - שמירת קובץ ישירות בתיקיה שמצביע לאותו inode



File links

In -s /Dir1/name3
/Dir3/name7



```
$touch
```

```
file1
```

Create 2 files

```
$touch
```

```
file2
```

```
$echo "Google" > file1
```

```
$echo "Microsoft" > file2
```

Add data to them

```
$cat file1
```

```
Google
```

```
$cat file2
```

```
Microsoft
```

Print the files content

```
$ln file1 file1-hardlink
```

```
$ln -s file2 file2-softlink
```

Create links to files

```
$ls -l
```

```
file1
```

```
file1-hardlink
```

```
file2
```

```
file2-softlink -> file2
```

Print dir content

Interesting Part

```
$mv file1 file1-new
```

```
$cat file1-hardlink
```

```
Google
```

Changing the name
of original has no effect

```
$mv file2 file2-new
```

```
$cat file2-softlink
```

```
cat: file2-softlink: No such file  
or directory
```

Changing the name
breaks the link

BUFFER CACHE

- עבודה עם הדיסק מתבצעת בבלוקים – מערכת ההפעלה יכולה לכתוב או לקרוא בלוק מסוים
- זה מציב בפנינו בעיה:
- נניח ואנו עורכים קובץ טקסט בזיכרון, כל שינוי בקובץ, לא משנה כמה קטן, ידרוש כתיבה מחדש של כל הבלוק
- כדי למנוע זאת, מערכת ההפעלה משתמשת במבנה הנקרא buffer cache
- ה buffer cache נמצא בזכרון (לא בדיסק) ושומר בתוכו חלק מהבלוקים של הזיכרון

BUFFER CACHE

- לכל בלוק בcache יש שני מאפיינים חשובים:
- חופשי או תפוס
- תפוס – תהליך מסויים עובד עם הבלוק
- חופשי – אף תהליך לא עובד איתו
- נקי או מלוכלך
- נקי – הבלוק זהה לבלוק בדיסק
- מלוכלך - יש הבדל בין הבלוק בcache לבלוק בדיסק

BUFFER CACHE

• כאשר תהליך מסויים צריך לגשת לבלוק מסויים בדיסק, מערכת ההפעלה בודקת את הcache

1. אם הבלוק נמצא

1. אם הוא חופשי – הבלוק מסומן כתפוס

2. אם הוא תפוס – התהליך מושהה עד שהבלוק יתפנה

2. אם הבלוק לא נמצא, מערכת ההפעלה תעבור על הבלוקים החופשיים, במטרה למצוא בלוק שאפשר להחריף בבלוק הנדרש

1. אם אין בלוקים חופשיים התהליך יושהה עד שיתפנה בלוק

2. אם יש, מערכת ההפעלה תתחיל לעבור בלוק אחרי בלוק

1. אם הבלוק מלוכלך (נעשו בו שינויים שעוד לא עודכנו בדיסק) – מערכת ההפעלה תסמן שיש לעדכן את הבלוק בדיסק, ותמשיך לבלוק הבא

2. כשימצא בלוק חופשי נקי היא תחליף אותו בבלוק הנדרש

שאלה 1

- נתון קובץ בכתובת `/usr/tmp/a.txt`
- ידוע ש:
- גודל הקובץ הוא בלוק יחיד
- `inode` של התיקיה הראשית אינו טעון לזיכרון
- הרשומות עבור `usr, tmp, a.txt` נמצאות בבלוק הראשון של התיקיה בה הם נמצאות
- כמה גישות לדיסק ידרשו כדי לקרוא את כל הנתונים בקובץ?

שאלה 1

- נתון קובץ בכתובת `/usr/tmp/a.txt`
- ידוע ש:
- גודל הקובץ הוא בלוק יחיד
- `inode` של התיקיה הראשית אינו טעון לזיכרון
- הרשומות עבור `usr`, `tmp`, `a.txt` נמצאות בבלוק הראשון של התיקיה בה הם נמצאות
- כמה גישות לדיסק ידרשו כדי לקרוא את כל הנתונים בקובץ?
- תחילה, אנו צריכים גישה אחת כדי לטעון את ה`inode` של התיקיה הרשאית, בתוכו נראה את הכתובות של הבלוק הראשון, נקרא גם אותו כדי למצוא את ה`inode` של תיקייה `usr`
- באופן דומה נצטרך שתי גישות עבור `usr` ושתי גישות עבור `tmp` כדי למצוא את מספר ה`inode` של `a.txt`
- עוד שתי גישות כדי לקבל את בלוק המידע של `a.txt`
- בסה"כ – 8 גישות

שאלה 2

- Inode מכיל 10 כתובות ישירות (direct block) וכתובת אחת לכל רמה של indirect (single, double, triple). גודל כתובת הוא 4byte
- מה הגודל המקסימלי האפשרי של קובץ בהינתן
 1. בלוק בגודל 1 kb
 2. בלוק בגודל 4kb

שאלה 2

- Inode מכיל 10 כתובות ישירות (direct block) וכתובת אחת לכל רמה של indirect (single, double, triple). גודל כתובת הוא 4byte
- מה הגודל המקסימלי האפשרי של קובץ בהינתן
א. בלוק בגודל 1 kb
- בלוקים ישירים – 10 בלוקים
- Single indirect – בלוק בגודל 1 kb, כתובת בגודל 4byte, $256 = \frac{1024}{4}$ – הבלוק מכיל 256 כתובות – 256 בלוקים נוספים
- באופן דומה Double יתן עוד 256^2 בלוקים וtriple 256^3 בלוקים
- כל בלוק הוא 1 Kb אז מקבלים בסה"כ
$$1kb \cdot (10 + 256 + 256^2 + 256^3) \approx 16gb$$

שאלה 2

- Inode מכיל 10 כתובות ישירות (direct block) וכתובת אחת לכל רמה של indirect (single, double, triple). גודל כתובת הוא 4byte
- מה הגודל המקסימלי האפשרי של קובץ בהינתן
2. בלוק בגודל 4kb
- בלוקים ישירים – 10 בלוקים
- Single indirect – בלוק בגודל 4kb, כתובת בגודל 4byte, $1024 = \frac{4 \cdot 1024}{4}$ – הבלוק מכיל 1024 כתובות – 1024 בלוקים נוספים
- באופן דומה Double יתן עוד 1024^2 בלוקים וtriple 1024^3 בלוקים
- כל בלוק הוא 4Kb אז מקבלים בסה"כ
$$4kb \cdot (10 + 1024 + 1024^2 + 1024^3) \approx 4tb$$

שאלה 3

- תהליך מסויים דורש גישה לקובץ `/tmp/file.txt` – ללא קריאה הנתונים שבו
- ה `inode` של התיקיה הראשית הראשית טעון לזיכרון
- אף `block` רלוונטי אחר אינו טעון
- האם ייתכן שתידרש קיראה של יותר מ-3 בלוקים?

שאלה 3

- תהליך מסויים דורש גישה לקובץ `/tmp/file.txt` – ללא קריאה הנתונים שבו
- `inode` של התיקיה הראשית הראשית טעון לזיכרון
- אף `block` רלוונטי אחר אינו טעון
- האם ייתכן שתידרש קיראה של יותר מ-3 בלוקים?

יתכן.

לדוגמא, אם גודל התוכן של הספריה `/tmp` הוא יותר מבלוק אחד וחיפוש כניסה עבור `/tmp/file.txt` דורש קריאה של שני הבלוקים.

במצב כזה יתכן שנקרא 4 בלוקים מהדיסק:

1. עבור גישה לטבלת ה `inodes` על מנת לקרוא את ה `inode` - של הספריה , `/tmp`

2. עבור קריאת תוכן הספריה `/tmp` ומציאת מספר ה `inode` של `/tmp/file.txt`

וגישה 1. נוספת לטבלת ה `inodes` על מנת לקרוא את ה `inode` של `/tmp/file.txt`

שאלה 4

inode מכיל 10 מצביעים של direct blocks. נתון שגודל בלוק הוא 1 KB (kilobyte) וגודל מצביע לבלוק הוא 4 B (4 bytes). נתון שה-inode של קובץ מסוים נמצא בזיכרון וגודל הקובץ גדול מ-500 KB. תהליך שפתח את הקובץ קורא לפונקציה read על מנת לקרוא את ה-500 KB הראשונים בקובץ. נתון שבמערכת יש buffer cache בגודל 100 MB (100 Megabytes) והוא ריק לפני הקריאה ל-read. כמה בלוקים יקראו מהדיסק כתוצאה של הקריאה ל-read?

א. 503

ב. 500

ג. 501

ד. 502

שאלה 4

inode מכיל 10 מצביעים של direct blocks. נתון שגודל בלוק הוא 1 KB (kilobyte) וגודל מצביע לבלוק הוא 4 B (4 bytes). נתון שה-inode של קובץ מסוים נמצא בזיכרון וגודל הקובץ גדול מ-500 KB. תהליך שפתח את הקובץ קורא לפונקציה read על מנת לקרוא את ה-500 KB הראשונים בקובץ. נתון שבמערכת יש buffer cache בגודל 100 MB (100 Megabytes) והוא ריק לפני הקריאה ל-read. כמה בלוקים יקראו מהדיסק כתוצאה של הקריאה ל-read?

א. 503

ב. 500

ג. 501

ד. 502

הקובץ ממופה ע"י 10 מצביעי direct, 256 מצביעים ב-single indirect ובלוק נוסף של מצביעים ב-double indirect. לכן מעבר ל-500 בלוקים של המידע של הקובץ שיקראו מהדיסק, יקראו גם 3 בלוקים של מצביעים (בלוק אחד עבור single indirect ושני בלוקים עבור double indirect).
(גם אם ניגשים לבלוקים של המצביעים מספר פעמים, תהיה רק גישה אחת לדיסק עבור כל בלוק כי ה-buffer cache גדול מספיק.)

שאלה 5

- במערכת קבצים של UNIX, נמחק בשל תקלה תוכן הקובץ של הספריה `/usr/ast/work`.
 - ספרייה זו אינה מכילה תתי ספריות, אלא רק 100 קבצים רגילים. נניח שידועים מספרי ה-inodes של הקבצים בספריה.
 - נניח שלכל קובץ בספריה בוצע לפני התקלה `softlink` מקובץ שאינו בספריה.
1. האם ניתן לשחזר את הרשימה של שמות כל הקבצים בספרייה?
 2. האם בהינתן שם של קובץ כלשהו בספריה ניתן לשייך לו את מספר ה-inode המתאים באופן ודאי?

שאלה 5

1. ניתן לשחזר את שמות הקבצים משום שה path-של קובץ היעד נשמר כאשר יוצרים אליו (softlink כתובנו של קובץ ה. (softlink-לפיכך ניתן לסרוק את עץ המחיצות ולמצוא softlinks אלו.
2. לא ניתן לשייך את שמות ה inodes-בתיקיה שנמחקה משום שה softlink-אינו שומר את ה- target inode.